INTERVIEW PREP SERIES

React

Interview Questions & Easy Answers

---

52 questions · 23 marked ★ most asked

Covers JSX, components, props and state, hooks (useState, useEffect, useContext, useRef, useMemo, useCallback, custom hooks), the virtual DOM, routing, Redux and performance. Matches what was reported in MERN fresher drives (Tech Mahindra, startups) and frontend rounds.

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Basics

Q1. What is React? ★ Most asked

React is an open-source JavaScript library made by Meta for building user interfaces, especially single-page applications. You build the UI from small reusable pieces called **components**, and React updates the page efficiently when data changes.

Q2. What are the main features of React? ★ Most asked

- Component-based architecture
- JSX syntax
- Virtual DOM for fast updates
- One-way data flow (parent to child via props)
- Hooks for state and side effects
- Huge ecosystem (React Router, Redux, Next.js)

Q3. What is JSX? ★ Most asked

JSX is a syntax that lets you write HTML-like code inside JavaScript. Babel converts it into `React.createElement()` calls.

```
const el = <h1 className="title">Hello, {name}</h1>;
```

Rules: return one parent element (or a Fragment), use `className` instead of `class`, close all tags, and put JS expressions inside `{ }`.

Q4. What is the Virtual DOM and how does it work? ★ Most asked

The Virtual DOM is a lightweight copy of the real DOM kept in memory. When state changes:

1. React creates a new virtual DOM tree.

2. It compares it with the previous one (this is called **diffing**).

3. It updates only the changed parts in the real DOM (this is **reconciliation**).

Updating the real DOM is slow, so touching only what changed makes React fast.

Q5. What is the difference between the real DOM and the virtual DOM?

The real DOM is what the browser shows; updating it is slow and causes re-layout. The virtual DOM is a JS object in memory; it is cheap to create and compare, and React uses it to work out the smallest set of real DOM updates.

Q6. What is a Single Page Application (SPA)?

An SPA loads one HTML page once and then updates the content with JavaScript, without full page reloads. Gmail and most React apps are SPAs. Pros: fast, app-like feel. Cons: slower first load and weaker SEO unless you use SSR (e.g. Next.js).

Components, props and state

Q7. What are components? What are the types? ★ Most asked

Components are reusable, independent pieces of UI, like a Navbar or a Card.

- **Functional components**: plain JS functions that return JSX; use hooks for state. This is the modern standard.

- **Class components**: ES6 classes that extend `React.Component`, use `this.state` and lifecycle methods. Older style.

Q8. What is the difference between props and state? ★ Most asked

- **Props** are inputs passed from parent to child. They are read-only for the child.
- **State** is data owned and managed inside the component. When state changes, the component re-renders.

```
function Greeting({ name }) {        // name is a prop
  const [count, setCount] = useState(0); // count is state
  return <p onClick={() => setCount(count + 1)}>{name}: {count}</p>;
}
```

Q9. What is the difference between class and functional components? ★ Most asked

Class components use `this.state`, `setState`, and lifecycle methods like `componentDidMount`. Functional components are simpler functions and use hooks (`useState`, `useEffect`) for the same features. Functional components have less code, no `this` confusion, and are recommended today.

Q10. What is prop drilling and how do you avoid it? ★ Most asked

Prop drilling is passing props through many layers of components that don't need them, just to reach a deep child. Avoid it with the **Context API**, a state library like **Redux** or **Zustand**, or by restructuring components (component composition).

Q11. What are keys in React and why are they important?

Keys are unique identifiers for list items. They help React know which items changed, were added or were removed, so it updates efficiently.

```
{users.map(u => <li key={u.id}>{u.name}</li>)}
```

Avoid using the array index as key if the list can change order, because it can cause bugs.

Q12. What is a React Fragment?

A Fragment lets you return multiple elements without adding an extra `<div>` to the DOM. Written as `<React.Fragment>...</React.Fragment>` or `<>...</>`.

Q13. What is conditional rendering?

Showing different UI based on a condition.

```
{isLoggedIn ? <Dashboard /> : <Login />}
{error && <p>{error}</p>}
```

Q14. What are controlled and uncontrolled components? ★ Most asked

In a **controlled** component, the form value lives in React state and changes through `onChange`. In an **uncontrolled** component, the DOM keeps the value and you read it with a `ref`.

```
const [email, setEmail] = useState("");
<input value={email} onChange={e => setEmail(e.target.value)} />
```

Controlled is preferred because React stays the single source of truth.

Q15. What is lifting state up?

When two sibling components need the same data, you move that state to their common parent and pass it down as props. This keeps them in sync.

Q16. What is one-way data binding in React?

Data flows only from parent to child through props. A child cannot directly change the parent's data; it calls a function passed by the parent instead. This makes data flow predictable and easier to debug.

Q17. What are synthetic events?

React wraps browser events in its own `SyntheticEvent` object so events behave the same in every browser. You use them like normal: `onClick`, `onChange`, `e.preventDefault()`.

Hooks

Q18. What are hooks? What are the rules of hooks? ★ Most asked

Hooks are functions that let functional components use state and other React features. Common ones: `useState`, `useEffect`, `useContext`, `useRef`, `useMemo`, `useCallback`, `useReducer`.

Rules:

- Call hooks only at the top level, never inside loops, conditions or nested functions.
- Call hooks only from React function components or custom hooks.

Q19. Explain `useState`. ★ Most asked

`useState` adds state to a functional component. It returns the current value and a function to update it.

```
const [count, setCount] = useState(0);
setCount(count + 1);
setCount(prev => prev + 1); // safer when new value depends on old
```

Updating state triggers a re-render. State updates are asynchronous and batched.

Q20. Explain `useEffect` and its dependency array. ★ Most asked

`useEffect` runs side effects like API calls, timers and subscriptions after the component renders.

```
useEffect(() => {
  fetch("/api/users").then(r => r.json()).then(setUsers);
}, []);
```

- No array: runs after every render.
- `[]` empty array: runs once after the first render (like componentDidMount).
- `[id]`: runs when `id` changes.
- The function you return is the **cleanup**, which runs before the next effect or on unmount (like componentWillUnmount).

```
useEffect(() => {
  const t = setInterval(tick, 1000);
  return () => clearInterval(t);
}, []);
```

Q21. What is the Context API and `useContext`? ★ Most asked

Context shares data (like theme, logged-in user, language) with many components without passing props manually at every level.

```
const ThemeContext = createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
```

```
      <Navbar />
    </ThemeContext.Provider>
  );
}

function Navbar() {
  const theme = useContext(ThemeContext); // "dark"
  return <nav className={theme}>...</nav>;
}
```

Interview tip: A Tech Mahindra candidate was asked to write useContext pseudo-code. Practise writing the 3 steps: create, provide, consume.

Q22. What is `useRef`? ★ Most asked

`useRef` returns an object `{ current: value }` that stays the same between renders. Changing it does NOT cause a re-render. Uses: accessing DOM elements (focus an input) and storing values like timer IDs or the previous value.

```
const inputRef = useRef(null);
<input ref={inputRef} />
inputRef.current.focus();
```

Q23. What is the difference between `useMemo` and `useCallback`? ★ Most asked

Both are for performance and both take a dependency array.

- `useMemo` caches a **computed value**: `const total = useMemo(() => calcTotal(items), [items]);`
- `useCallback` caches a **function** so it is not recreated on every render: `const handle = useCallback(() => {...}, [id]);`

Use them only when there is a real performance problem.

Q24. What is `useReducer`?

An alternative to `useState` for complex state with many actions. You write a reducer function `(state, action) => newState` and call `dispatch({ type: "add" })`. It is similar to how Redux works.

Q25. What is a custom hook? ★ Most asked

A custom hook is your own function starting with `use` that reuses stateful logic across components.

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    fetch(url).then(r => r.json()).then(d => { setData(d); setLoading(false); });
  }, [url]);
  return { data, loading };
}
```

Q26. What is the difference between `useEffect` and `useLayoutEffect`?

`useEffect` runs after the browser paints the screen (non-blocking). `useLayoutEffect` runs after DOM changes but before painting; use it only when you must measure or change the layout without a visible flicker.

Q27. Why should you not update state directly?

Writing `this.state.count = 5` or mutating an array in state does not tell React anything changed, so the UI won't re-render. Always use the setter and create new objects/arrays: `setItems([...items, newItem])`.

Lifecycle and rendering

Q28. What are the lifecycle methods of a class component? ★ Most asked

- **Mounting**: `constructor`, `render`, `componentDidMount` (API calls go here).
- **Updating**: `shouldComponentUpdate`, `render`, `componentDidUpdate`.
- **Unmounting**: `componentWillUnmount` (cleanup).

In functional components, `useEffect` covers all three.

Q29. When does a React component re-render? ★ Most asked

When its state changes, when its props change, when its parent re-renders, or when a context it uses changes.

Q30. What is `React.memo`?

`React.memo` wraps a functional component so it re-renders only when its props actually change. It is like `PureComponent` for class components.

```
const Card = React.memo(function Card({ title }) { return <h3>{title}</h3>; });
```

Q31. What is a Higher-Order Component (HOC)?

An HOC is a function that takes a component and returns a new component with extra features, e.g. `withAuth(Dashboard)` that checks login first. Hooks have replaced most HOC use cases.

Q32. What is reconciliation and the diffing algorithm?

Reconciliation is how React decides what to change in the DOM. The diffing algorithm assumes: elements of different types produce different trees (so it replaces them), and keys identify list items. This makes comparing trees fast (O(n)).

Q33. What is React Fiber?

Fiber is React's internal engine (since React 16). It splits rendering work into small units, so React can pause, prioritise and resume work, which keeps the app responsive during big updates.

Q34. What is StrictMode?

`<React.StrictMode>` is a development-only tool that highlights potential problems. It renders components twice in development to catch side-effect bugs. It does nothing in production.

Routing, forms and data

Q35. What is React Router? How do you set up routes? ★ Most asked

React Router handles navigation between pages in an SPA without reloading.

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/user/:id" element={<User />} />
```

```
    <Route path="*" element={<NotFound />} />
  </Routes>
</BrowserRouter>
```

Use `<Link to="/about">` instead of `<a>`, `useParams()` to read `:id`, and `useNavigate()` to redirect in code.

Q36. How do you call an API in React? ★ Most asked

Use `fetch` or `axios` inside `useEffect`, store the result in state, and handle loading and errors.

```
useEffect(() => {
  axios.get("/api/products")
    .then(res => setProducts(res.data))
    .catch(err => setError(err.message))
    .finally(() => setLoading(false));
}, []);
```

Q37. How do you handle forms in React?

Use controlled inputs with state, an `onSubmit` handler with `e.preventDefault()`, and validation. For big forms, libraries like **React Hook Form** or **Formik** help.

Q38. How do you implement protected routes?

Create a wrapper component that checks if the user is logged in (from context or Redux). If yes, render the page; if not, redirect with `<Navigate to="/login" />`.

State management

Q39. What is Redux and why use it? ★ Most asked

Redux is a library to manage global state in one central **store**. It is useful when many components share data. Core ideas:

- **Store**: holds the whole state.
- **Action**: an object describing what happened, `{ type: "cart/add", payload: item }`.
- **Reducer**: a pure function that returns new state based on the action.
- **Dispatch**: sends an action to the store.

Today, **Redux Toolkit** is the recommended way; it reduces boilerplate with `createSlice` and `configureStore`.

Q40. What is the difference between Context API and Redux?

Context is built-in and good for small amounts of rarely-changing global data (theme, user). Redux is better for large apps with complex, frequently-updated state, and gives devtools, middleware and predictable patterns.

Q41. What are `useSelector` and `useDispatch`?

Hooks from react-redux. `useSelector(state => state.cart.items)` reads data from the store. `useDispatch()` returns the dispatch function to send actions.

Performance and advanced

Q42. How do you optimise a React app's performance? ★ Most asked

- Use `React.memo`, `useMemo` and `useCallback` to avoid unnecessary re-renders.
- Use proper keys in lists.
- Lazy-load components with `React.lazy` and `Suspense` (code splitting).
- Virtualise long lists (react-window).
- Keep state as local as possible.
- Optimise images and use the production build.

Q43. What is lazy loading / code splitting in React?

Loading a component only when it's needed, instead of in the first bundle.

```
const Admin = React.lazy(() => import("./Admin"));
<Suspense fallback={<p>Loading...</p>}><Admin /></Suspense>
```

Q44. What are error boundaries?

Components that catch JavaScript errors in their child tree and show a fallback UI instead of crashing the whole app. They must be class components using `componentDidCatch` or `getDerivedStateFromError` (or use the react-error-boundary library).

Q45. What are portals?

`createPortal(child, domNode)` renders a child into a different DOM node outside the parent, which is useful for modals, tooltips and popups that must appear above everything.

Q46. What is the difference between React and Angular?

React is a library focused on the UI layer, uses JSX and one-way data flow, and you pick other tools yourself. Angular is a full framework by Google with TypeScript, two-way binding, dependency injection and built-in routing.

Q47. What is Create React App vs Vite?

Both create a React project setup. CRA (now deprecated) used Webpack. **Vite** is the modern choice: it uses native ES modules in development, so it starts and reloads much faster.

Q48. What is the difference between server-side rendering (SSR) and client-side rendering (CSR)?

In CSR, the browser gets an almost empty HTML and JS builds the page, so the first view is slower and SEO is weaker. In SSR, the server sends fully built HTML, so the first view is faster and SEO is better. Next.js adds SSR to React.

Q49. What are React Server Components?

Components that run only on the server (React 19, Next.js App Router). They can fetch data directly and send ready HTML with zero JS to the browser, which makes pages lighter. Interactive parts are marked as Client Components with `"use client"`.

More frequently asked

Q50. What is the difference between state and props in terms of who can change them? ★ Most asked

Props are owned by the parent and are read-only in the child. State is owned by the component itself and changed with its setter. If a child must change parent data, the parent passes down a callback function as a prop.

Q51. Why does `setState` / `setCount` not update the value immediately?

State updates are scheduled and batched for performance, so reading the variable right after calling the setter still shows the old value. Use the updater form `setCount(c => c + 1)` when the new value depends on the old one, or `useEffect` to react after the update.

Q52. What is the `children` prop?

Whatever you put between a component's opening and closing tags is passed as `props.children`. It's used to build wrapper components like `<Card>`, `<Modal>` or layouts.

```
function Card({ children }) { return <div className="card">{children}</div>; }
<Card><h2>Title</h2></Card>
```